

# Unit V: Access Control

5.1 Access Control Principles

5.2 Subjects, Objects and Access Rights

5.3 Access Control Matrix and Compatibility Lists

5.4 Discretionary Access Control

5.5 Role Based Access Control

5.6 Attribute Based Access Control

5.7 Identity, Credential and Access Management

5.8 Trust Framework

- **Access control** refers to the process of managing and regulating **who has access to specific resources or information** within an organization.
- It involves determining **who is authorized to access certain information** or resources, and under what conditions they can do so.
- Access control is important for **maintaining the security and confidentiality of sensitive information and resources** within an organization.
- It helps **prevent unauthorized access, modification, or destruction** of data, which can lead to data breaches, identity theft, or other security incidents.
- Access control can also help organizations comply with regulations and industry standards, such as HIPAA or PCI-DSS.

# several requirements for effective access control

- **Identification and authentication of users:** Users must be **uniquely identified and authenticated** before being granted access to resources or information.
- **Authorization:** Users must be **authorized to access specific resources** or information based on their roles and responsibilities within the organization.
- **Accountability:** Access control systems should **log and monitor all access attempts and actions** taken by users.
- **Segregation of duties:** Access control policies should ensure that **no one individual has too much control or influence over a particular resource** or information.
- **Least privilege:** Users should only be granted the **minimum level of access necessary to perform their job** responsibilities.

# Modern access control systems

- In modern access control systems, there are several technologies and approaches used to control access to resources and information.
- These include **biometric authentication, multi-factor authentication, role-based access control, attribute-based access control**, and others.
- Modern access control systems often rely on a combination of these technologies to provide **robust security and access control capabilities**.



# HIPAA and PCI-DSS

- HIPAA (Health Insurance Portability and Accountability Act) is a federal law in the United States that sets standards for the **protection of sensitive patient health information**.
- It establishes rules for **healthcare providers, health plans, and healthcare clearinghouses to ensure the confidentiality, integrity, and availability of electronic protected health information (ePHI)**.
- Compliance with HIPAA regulations is **mandatory for all covered entities** and their business associates.
- PCI-DSS (Payment Card Industry Data Security Standard) is a set of **security standards** developed by **major credit card companies to protect against credit card fraud** and ensure the security of cardholder data.
- The PCI-DSS standards cover all aspects of payment card processing, including **data storage, transmission, and processing**.
- Compliance with the PCI-DSS standards is **mandatory for any organization that accepts credit card payments**. Non-compliance with the PCI-DSS standards can result in **penalties, fines, and legal action**.

# Basic security and derived security

- Examples of basic security requirements include:
- **Authentication and Authorization** - ensuring that only authorized users have access to sensitive information and resources.
- **Encryption** - ensuring that sensitive data is protected from unauthorized access by using encryption technologies.
- **Access control** - ensuring that access to sensitive resources is restricted based on the principle of least privilege.
- Examples of derived security requirements include:
- **Backup and recovery** - ensuring that critical data is backed up regularly and can be recovered in case of a disaster.
- **Monitoring and logging** - ensuring that system activity is monitored and logged for auditing and investigation purposes.
- **Physical security** - ensuring that physical access to sensitive resources is restricted and monitored.
- While basic security requirements are essential for any security program, derived security requirements are equally important, as they help to ensure that the unique security needs of a system or organization are met.

# Access Control Context

- Access control is the process of ensuring that only authorized entities are able to access specific resources or information in a system or organization.
- Access control can be applied at different levels, such as **physical access control, network access control, and application access control**.
- The access control context refers to the overall environment in which access control is applied, and includes various factors such as the type **of resources being protected**, the security needs of the organization, and the **specific access control policies** in place.

# Access Control Context

## Authentication:

- **Verifies the identity of a user** or entity attempting to access a system or resource.
- Can be achieved through various methods such as **username and password, biometric identification, smart card authentication, or two-factor authentication.**
- Ensures that **only authorized users** are able to access the system or resource.

## Authorization:

- Determines **what actions a user or entity is allowed to perform** once they have been authenticated.
- Based on the user's identity and the specific access control policy in place.
- Implemented through various methods such as **access control lists (ACLs), roles and permissions, or attribute-based access control (ABAC).**

## Audit:

- **Records and monitors access** attempts to a system or resource.
- Provides a record of **who has accessed a resource, what actions they have taken, and when these actions** were performed.
- Important **for detecting and investigating security breaches**, as well as for compliance and regulatory purposes.

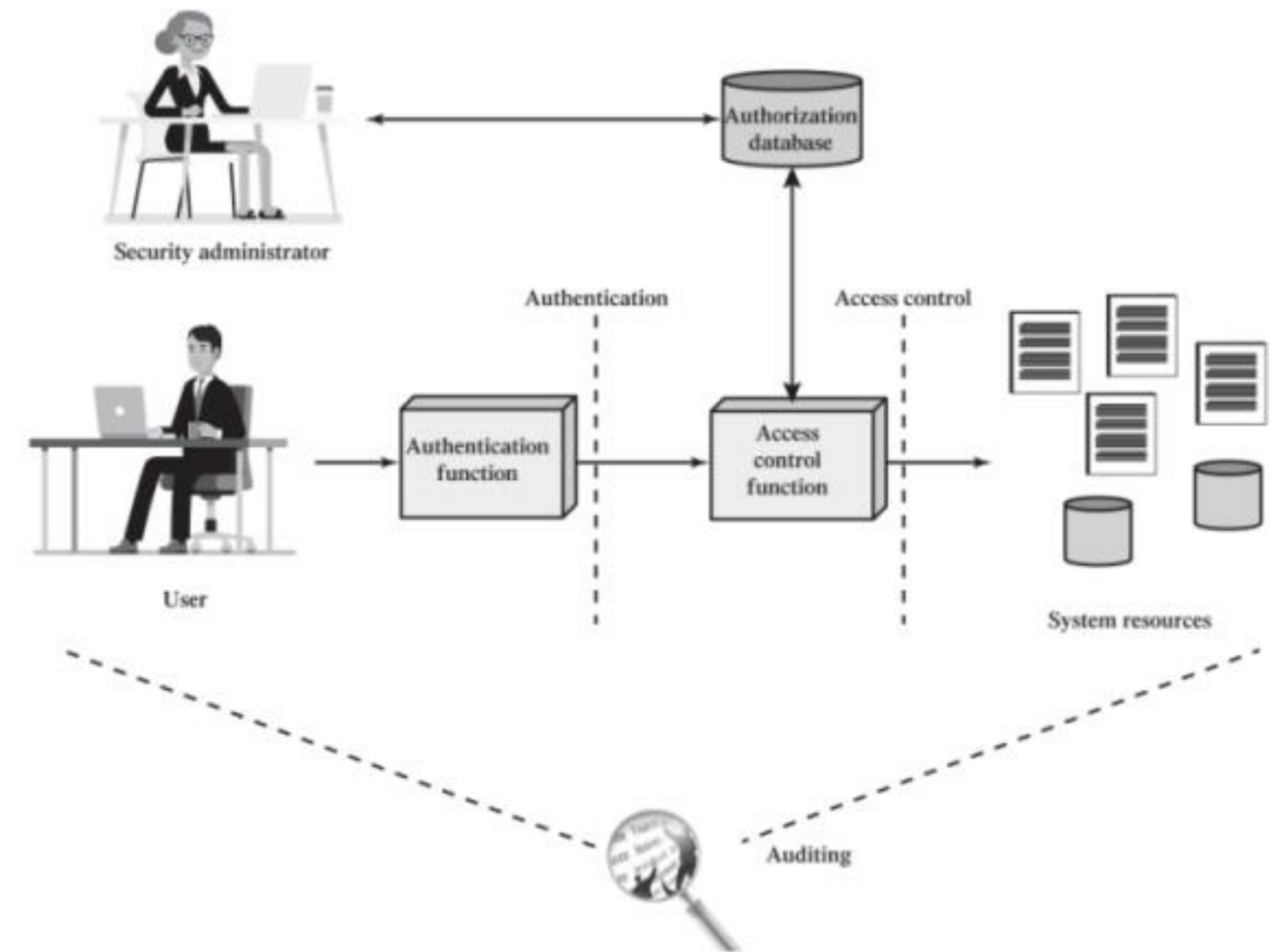


Figure 4.1 Relationship Among Access Control and Other Security Functions



# Access control policies

- Access control policies are a **set of rules and procedures that define how access to resources** or information should be granted or denied.
- Access control policies are designed to **ensure that only authorized entities are able to access specific resources** or information in a system or organization.
- Access control policies can be based on various access control models, such as
  1. Discretionary Access Control (DAC):
    - A user creates a **file** and **sets permissions** to **allow only specific users or groups** to access it.
    - A user shares a **folder** on a network and **grants read access to their colleagues**, but **denies access to external users**.
  2. Mandatory Access Control (MAC):
    - A government agency uses **security clearances to determine** access to classified information. Only users with the appropriate clearance level can access specific information.
    - A financial institution uses a **security classification system to control access to sensitive customer data**. Only employees with a need-to-know and appropriate clearance level can access the data.

# Access control policies

## Role-Based Access Control (RBAC):

- An online store uses **RBAC to grant permissions to users based on their roles.**
- Customers have **access to the shopping cart and order history**, while **employees have access** to additional functionality such as **product management and customer service.**
- A healthcare organization uses RBAC to **grant access to patient records** based on the **user's job function.**
- **Nurses** can access **patient charts and medication lists**, while **doctors can also access lab results and imaging reports.**
- Attribute-Based Access Control (ABAC):
- An airport security system uses ABAC to **grant access to restricted areas based on the user's security clearance level**, employment status, and location. **Only users with the appropriate attributes can access the area.**
- An e-commerce website uses ABAC to **grant access to promotional discounts** based on the **user's age, location, and purchase history.** Only users with the **appropriate attributes** can access the discount.

# Subjects, Objects and Access Rights

- In the context of access control, subjects, objects, and access rights are key components that help determine whether an entity is authorized to access a particular resource or perform a specific action. Here's what each of these terms means:
- **Subjects:** A subject is an entity that is attempting to access a resource or perform an action. Subjects can be **users, processes, devices, or any other entity that interacts with a system**. Example of subjects:
  - A **user** logging into a computer system
  - A **process** running on a server
  - A **device connected** to a network
  - A **third-party application** accessing an API

**Three classes of subjects: owner, group, and world.**

**Owner:** The owner class refers to the user who **owns the file or directory**. The **owner has full access to the file or directory** and can **modify its permissions**.

**Group:** The group class refers to a group of users **who have been granted access to the file or directory**. The group can have **different levels of access, such as read-only or read-write**, depending on the permissions set by the owner.

**World:** The world class refers to **all other users who are not part of the owner or group classes**. The world can also have different levels of access, such as **read-only or no access**, depending on the permissions **set by the owner**.

# Subjects, Objects and Access Rights

- Objects: An object is a **resource or information** that a subject is attempting to access or modify. Objects can include files, databases, servers, applications, and other resources. Examples
  - A **file** stored on a file server
  - A **database** containing customer information
  - A server hosting a **web application**
  - A **physical space**, such as a restricted area in a building

**Access Rights:** Access rights define the level of access that a subject has to an object.

- Access rights can include permissions such as **read, write, execute**, or delete, and can be assigned based on various factors such as the subject's role or job function, the sensitivity of the information, or the specific requirements of the system or organization. **Read access** to a file, allowing a user to **view** its contents
  - Write access to a database, allowing a user to **modify or add data**
  - Execute access to a server, allowing a user to **run specific commands or programs**
  - Entry access to a physical space, allowing a **user to enter a restricted area**



# An Access Control Matrix (ACM)

- An Access Control Matrix (ACM) is a **security model** used in computer systems **to enforce access control policies**.
- It is a table that describes the **relationship between subjects** (users, processes, or devices) and **objects** (resources such as files, directories, or network resources) in terms of **access rights or permissions**.
- In an Access Control Matrix, each **row represents an object**, and each **column represents a subject**. The **cells in the matrix indicate the access rights of each subject to each object**. The access rights can be expressed using different permissions such as **Read, Write, Execute, Delete**, or any other relevant permissions.

|          |        | OBJECTS              |                      |                      |                      |
|----------|--------|----------------------|----------------------|----------------------|----------------------|
|          |        | File 1               | File 2               | File 3               | File 4               |
| SUBJECTS | User A | Own<br>Read<br>Write |                      | Own<br>Read<br>Write |                      |
|          | User B | Read                 | Own<br>Read<br>Write | Write                | Read                 |
|          | User C | Read<br>Write        | Read                 |                      | Own<br>Read<br>Write |

(a) Access matrix

# Compatibility Lists

- Compatibility lists are used to define the **access** rights of a subject to a **particular object**.
- Each **object** has a **list of subjects** who are **authorized to access** it, along with their access rights.
- For example, a file may have a compatibility list that specifies which **users or groups are allowed to read, write, or execute the file**.

# An Access Control Matrix and Compatibility Lists

An Access Control Matrix

|       | File 1 | File 2 |
|-------|--------|--------|
| Alice | rwX    | rw     |
| Bob   | r-X    | rwX    |

Compatibility Lists

|      | File 1     | File 2              | File 3         |
|------|------------|---------------------|----------------|
| List | Alice, Bob | Alice, Bob, Charlie | Alice, Charlie |

# Discretionary Access Control (DAC)

- Discretionary Access Control (DAC) is a type of **access control system** where the **owner of a resource has the ability to grant or restrict access to that resource at their discretion** (will). In other words, the owner has the discretion to control access to the resource.
- DAC is commonly used in **operating systems and network environments** to protect resources such as **files, directories, and network shares**.
- Under DAC, access control is based on the **identity of the user or group** requesting access and the permissions granted to them **by the owner**.
- Access permissions are usually defined in terms of **read, write, and execute**, and may be **granted or revoked at any time by the owner**. The access permissions are stored as **metadata with the resource** and are enforced by the operating system or access control software.
- The key features of DAC are:
  - **Owner Control**: The owner of a resource has complete control over who can access it and what level of access they have.
  - **Permission-Based**: Access control is based on permissions granted by the owner, which can be modified as necessary.
  - **Flexibility**: DAC provides flexibility to the owner to set access controls based on the specific requirements of the resource.
  - **Simple**: DAC is a simple and easy-to-implement access control mechanism, making it a popular choice in many operating systems and applications.



# Discretionary Access Control (DAC)

|    | Object    | Operations  | User Role     | Resource Sensitivity | Time of Day    | Location     |
|----|-----------|-------------|---------------|----------------------|----------------|--------------|
| S1 | File1     | Read, Write | Administrator | High                 | Anytime        | Office A     |
| S2 | File1     | Read        | Manager       | Low                  | Business Hours | Office A     |
| S3 | File1     | Read        | Employee      | Low                  | Anytime        | Any Location |
| S4 | File2     | Read        | Employee      | Low                  | Anytime        | Any Location |
| S4 | Database1 | Read, Write | Manager       | High                 | Business Hours | Office B     |
| S5 | Database1 | Read        | Employee      | Low                  | Anytime        | Any Location |

Extended access control matrix

# Role-Based Access Control

- Role-Based Access Control (RBAC) is a model for **controlling access to resources** based on the **roles of users or entities within an organization**.
- RBAC is widely used in various industries including **healthcare, finance, and government**.
- The key idea of RBAC is to **separate the roles of users from their individual identities**.
- In RBAC, access to resources is determined based on the **roles that users are assigned to**.
- **Roles are defined by the organization** and are based on **job functions or responsibilities**.
- Users are then assigned **to one or more roles**, and the access rights of those roles are applied to the user.
- For example, a **nurse** may be assigned to the role of "**nurse**", which grants access to **patient records and medication orders**.

# Role-Based Access Control(primary components)

- RBAC typically involves three primary components: roles, permissions, and users.
- **Roles:** A role is a collection of permissions that define a specific job function or responsibility. Examples of roles in a **healthcare setting may include "doctor", "nurse", and "administrator"**. Each role has a set of permissions associated with it that dictate what actions can be performed by users assigned to that role.
- **Permissions:** Permissions define the actions that can be taken on a resource. Examples of permissions may **include "read", "write", "create", "delete", or "execute"**. Permissions are assigned to roles, and users inherit those permissions when they are assigned to a role.
- **Users:** **Users are assigned to one or more roles** based on their job function or responsibilities. Users inherit the permissions associated with those roles, and are able to perform the actions defined by those permissions.

# Role-Based Access Control

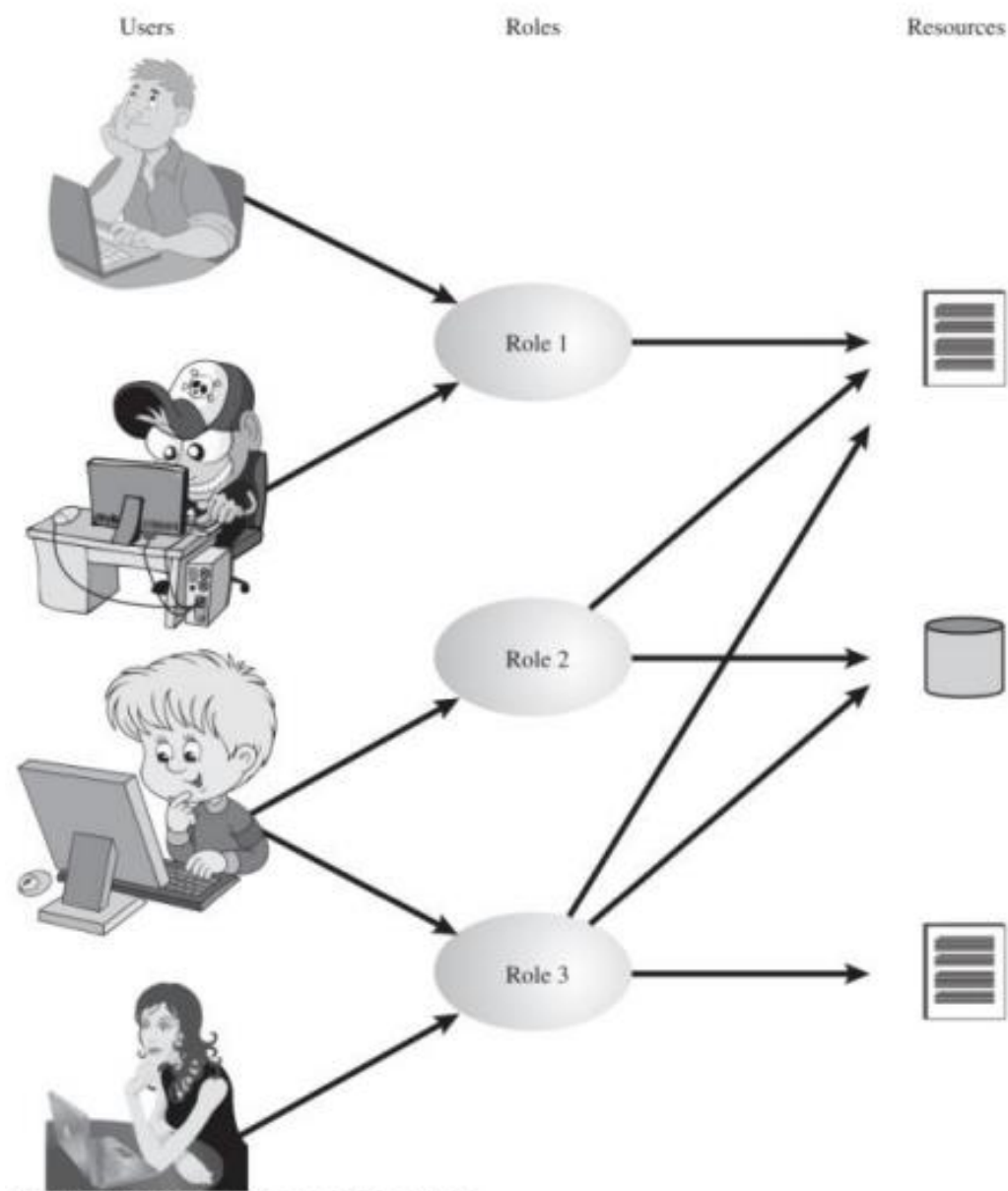


Figure 4.6 Users, Roles, and Resources

Objects

|          | $R_1$ | $R_2$ | $\dots$ | $R_n$ |
|----------|-------|-------|---------|-------|
| $U_1$    | ✕     |       |         |       |
| $U_2$    | ✕     |       |         |       |
| $U_3$    |       | ✕     |         | ✕     |
| $U_4$    |       |       |         | ✕     |
| $U_5$    |       |       |         | ✕     |
| $U_6$    |       |       |         | ✕     |
| $\vdots$ |       |       |         |       |
| $\vdots$ |       |       |         |       |
| $U_m$    | ✕     |       |         |       |

Roles

|            | Admin | Manager | Supervisor | Employee | Guest |
|------------|-------|---------|------------|----------|-------|
| File A     | RWX   | RWX     | RWX        | RW       | R     |
| File B     | RW    | RW      | RW         | R        | R     |
| File C     | RWX   | RW      | R          | R        | R     |
| Folder D   | RWX   | RWX     | RW         | R        | -     |
| Database E | RWX   | RWX     | R          | -        | -     |
| System F   | RWX   | R       | -          | -        | -     |

Access Controlled Matrix in RBAC



# RBAC implementation

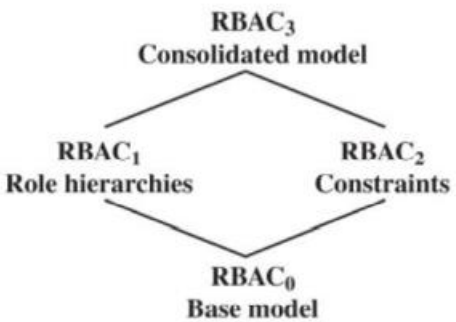
- **Hierarchical RBAC:** This approach involves defining roles in a hierarchical manner, with **higher-level roles inheriting permissions from lower-level roles**. For example, a "supervisor" role may inherit permissions from a "worker" role.
- **Constrained RBAC:** This approach involves **defining constraints on the roles** that users can be assigned to, based on **attributes such as job title, department, or location**.
- **Static RBAC:** This approach involves **defining roles and permissions in advance, and assigning users** to those roles based on their job function or responsibilities. This approach is often **used in smaller organizations with less complex security requirements**.
- **Dynamic RBAC:** This approach involves **dynamically assigning roles and permissions based on a user's behavior or context**. For example, a user may be granted **temporary access to a resource based on their location or time of day**.

# RBAC Reference Models

- A variety of functions and services can be included under the general RBAC approach. To clarify the various aspects of RBAC, it is useful to define a set of abstract models of RBAC functionality.
- There is a family of reference models that has served as the basis for ongoing standardization efforts. This family consists of four models that are related to each other, as shown in Figure.

1. **RBAC0** contains the **minimum functionality** for an RBAC system.
2. **RBAC1** includes the RBAC0 functionality and **adds role hierarchies**, which enable one role to inherit permissions from another role.
3. **RBAC2** includes RBAC0 and **adds constraints**, which restrict the ways in which the components of an RBAC system may be configured.
4. **RBAC3** contains the **functionality of RBAC0, RBAC1 and RBAC2**

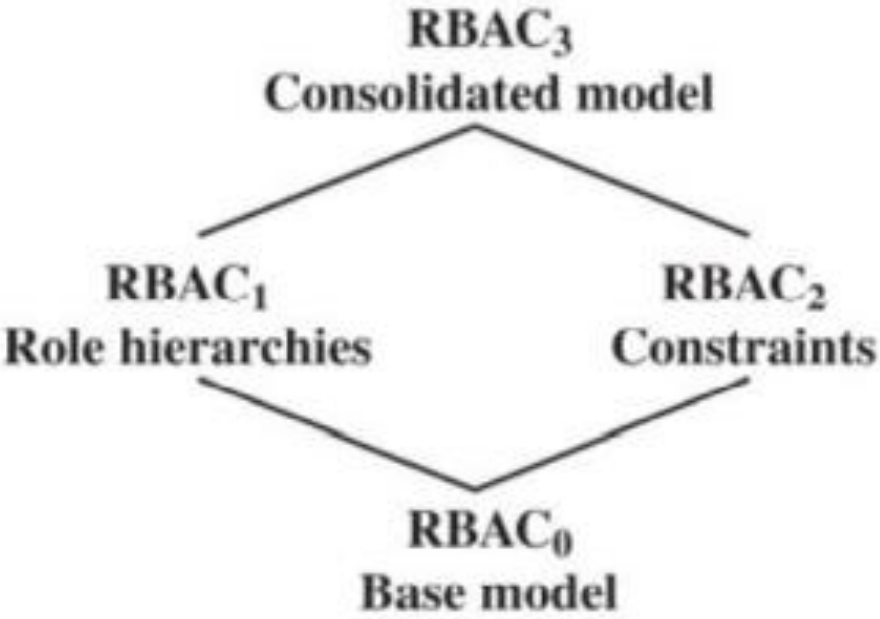
| Models | Hierarchies | Constraints |
|--------|-------------|-------------|
| RBAC0  | No          | No          |
| RBAC1  | Yes         | No          |
| RBAC2  | No          | Yes         |
| RBAC3  | Yes         | Yes         |



(a) Relationship among RBAC models

# RBAC Reference Models

| Models | Hierarchies | Constraints |
|--------|-------------|-------------|
| RBAC0  | No          | No          |
| RBAC1  | Yes         | No          |
| RBAC2  | No          | Yes         |
| RBAC3  | Yes         | Yes         |

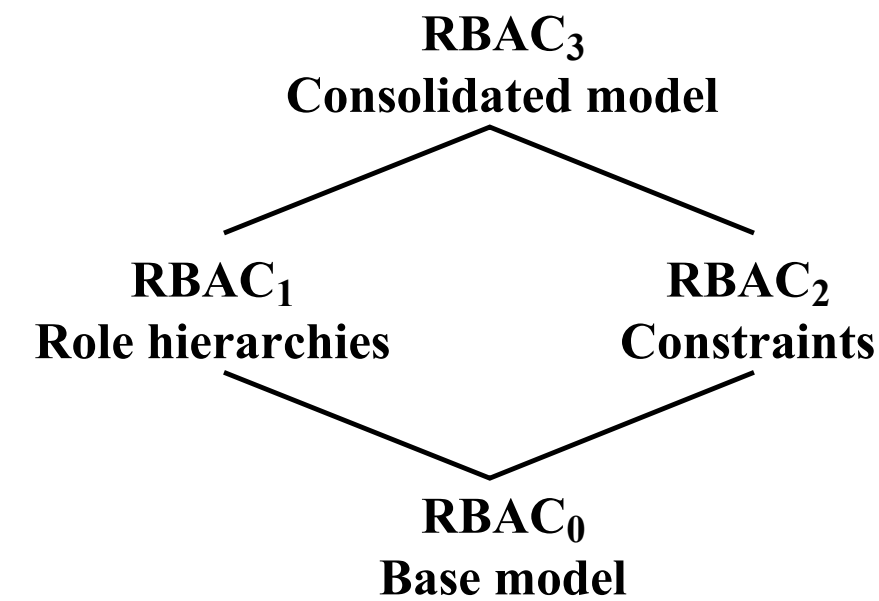


(a) Relationship among RBAC models

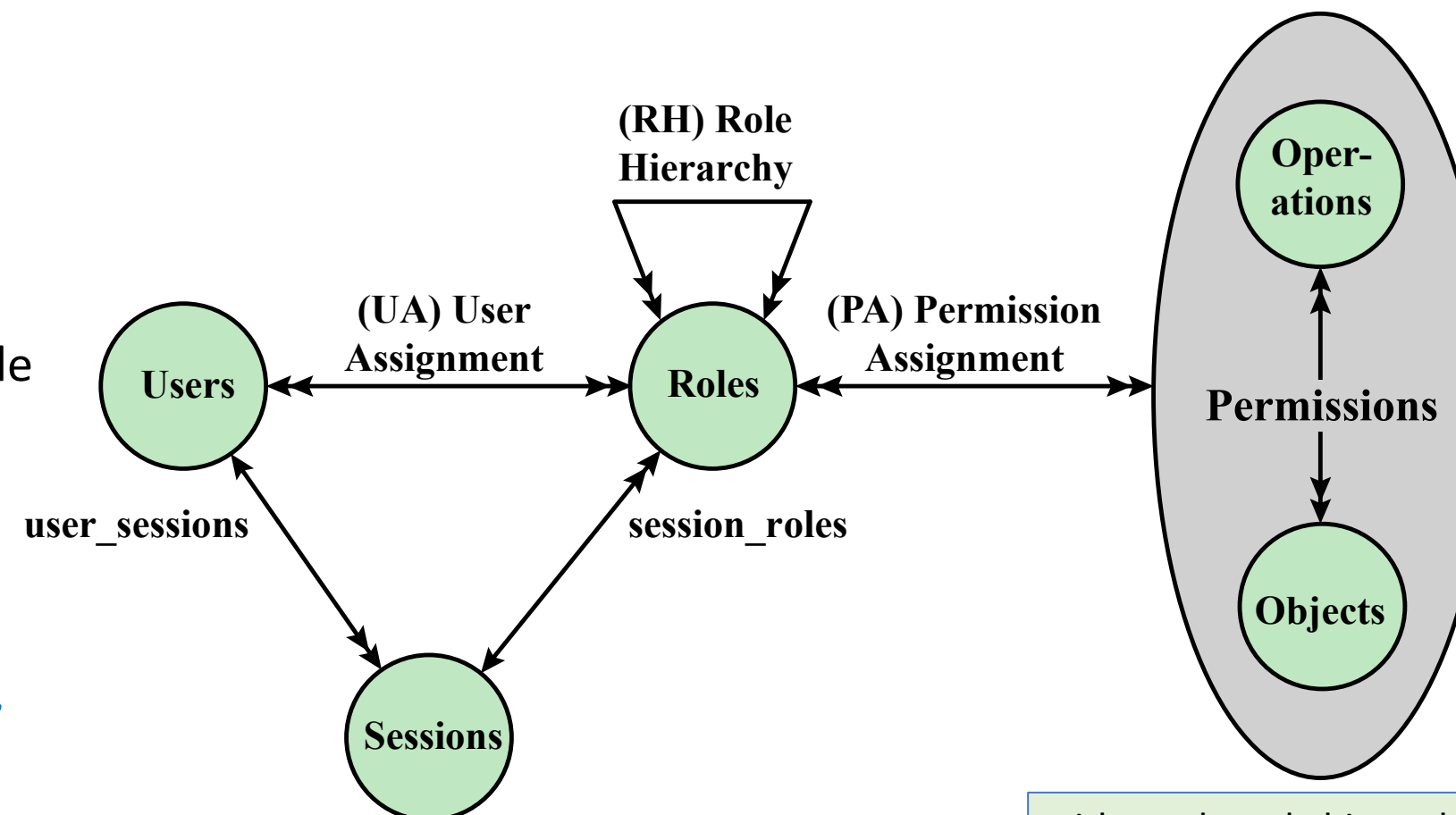


# Abstract models of RBAC functionality

- **Role**: A named **job** function within the organization that controls this computer system. (*authority & responsibility*)
- **Permission**: An **approval** of a particular mode of **access** to one or more objects. (*access right, privilege, authorization*).
- **Session**: A mapping between a **user** and an **activated subset of the set of roles** to which the user is assigned.
  - **Temporary** one-to-many relationship
  - **Least privilege**: Only needed roles
- One user may have multiple roles, and multiple users may be assigned to a single role (*many-to-many*).
- Flexibility and granularity: the **many-to-many** relationships between users and roles and between roles and permissions (*not found in conventional DAC schemes*).
  - Without this flexibility and granularity, there is a risk that a user may be granted more access to resources than is needed



(a) Relationship among RBAC models



(b) RBAC models

Figure 4.8 A Family of Role-Based Access Control Models.



# Attribute-Based Access Control (ABAC)

- Attribute-Based Access Control (ABAC) is a model that **uses attributes** to make access control decisions.
- It is based on the idea that access to resources should be granted based on the **attributes of the subjects (users)**, objects (resources), and **other contextual factors**.
- The attributes can include any information that can be used to make an access decision, such as user **roles**, group **membership**, **department**, **location**, **time of day**, **device**, and **other relevant factors**.
- ABAC is a dynamic and flexible model that can be customized to fit different types of systems and scenarios.
- It can provide more fine-grained access control than other models such as DAC and RBAC.
- ABAC can also be used to **enforce complex policies** that are difficult to implement with other models.

# Key element of Attribute-Based Access Control

- **Attributes:** Attributes are characteristics or properties of **subjects, objects, and the environment** that are used in determining whether access should be granted or denied. Examples of attributes include user **roles, location, time of day, device type, and data classification**.
  - **Subject attributes** refer to the characteristics or properties of the user or entity requesting access to a resource. Examples of subject attributes include **job title, department, security clearance level, or role**.
  - **Object attributes** refer to the characteristics or properties of the resource being accessed. Examples of object attributes include the sensitivity **level of a document, the location of a file**, or the type of data being accessed.
  - Environment attributes refer to the contextual information that can affect access decisions. Examples of environment attributes **include time of day, location, network security status, or the presence of other users**.

# Key element of Attribute-Based Access Control

- **Policies:** Policies define **the rules** that determine whether access should be granted or denied based on the attributes associated with the subject, object, and environment. Policies can be **simple or complex**, and can involve multiple attributes and conditions.
- **Policy Decision Point (PDP):** The PDP is the engine that **evaluates policies** and makes access control decisions based on the attributes associated with the **subject, object, and environment**.
- **Policy Enforcement Point (PEP):** The PEP is the mechanism that **enforces the access control decisions made by the PDP**. The PEP can be integrated with various systems and applications, such as **firewalls, web servers, and cloud-based services**.

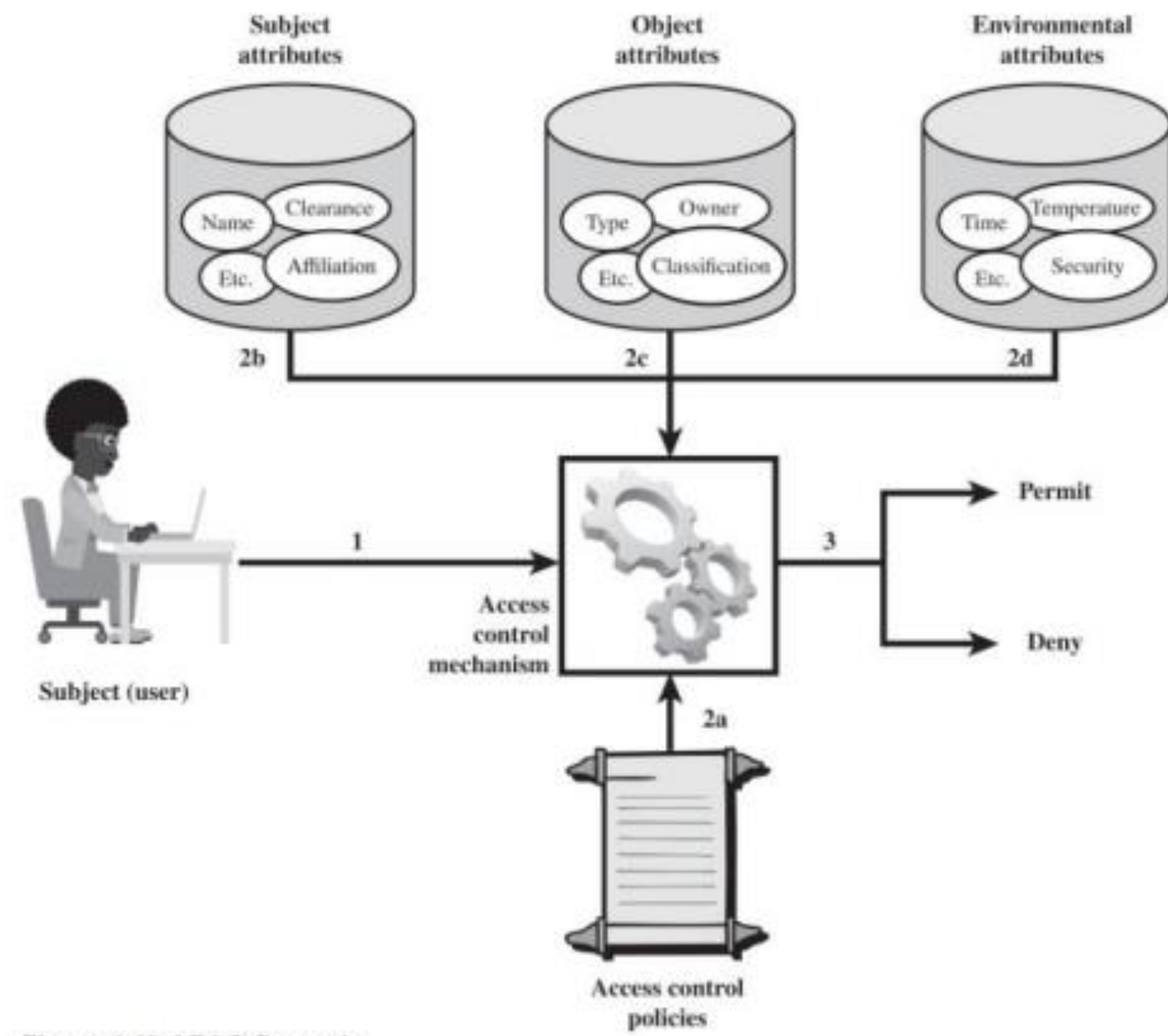


Figure 4.10 ABAC Scenario



# For example

- In ABAC system for a **financial institution**, **attributes** might include the **user's job title, the location of the user, the time of day, and the type of transaction being requested.**
- **Policies** might be defined to allow managers to approve transactions over a **certain amount, or to restrict access to certain types of transactions** based on the user's job title.
- The **PDP** would evaluate the attributes associated with the **user, the transaction, and the environment**, and make a decision on whether to grant or deny access.
- The **PEP** would then enforce the access control decision by either **allowing or denying the transaction.**

# Identity, Credential, and Access Management (ICAM)

- Identity, Credential and Access Management (ICAM) is a framework used by organizations to manage digital identities and access to resources.
- ICAM is based on a set of standards and guidelines developed by the National Institute of Standards and Technology (**NIST**) to help organizations manage identity, authentication, authorization, and access control.
- The ICAM framework provides a comprehensive approach to managing digital identities and access to resources. It helps organizations ensure that only authorized individuals have access to resources, while also protecting the privacy of individuals' personal information.
- ICAM is designed to help organizations manage digital identities across multiple systems and applications, while ensuring that access is only granted to authorized individuals. The framework includes the following components:
  - 1. Identity Management:** Identity management is the process of managing digital identities, including user authentication, user provisioning, and user de-provisioning. The goal of identity management is to ensure that only authorized individuals have access to resources.

# Identity, Credential, and Access Management (ICAM)

2. **Credential Management:** Credential management is the process of managing digital credentials, such as passwords, tokens, and digital certificates. The goal of credential management is to ensure that users are properly authenticated before they are granted access to resources.
3. **Access Management:** Access management is the process of controlling access to resources based on user identity and permissions. The goal of access management is to ensure that users only have access to the resources they need to perform their job functions.
4. **Federation:** Federation is the process of linking identity management systems between organizations to allow users to access resources across multiple organizations. Federation can simplify the process of managing access across multiple systems and organizations.
5. **Privacy:** Privacy is an important component of ICAM. Organizations must ensure that they are protecting the privacy of individuals' personal information, while also ensuring that authorized individuals have access to the resources they need.
6. **Risk Management:** Risk management is an important part of ICAM. Organizations must identify and manage risks associated with identity and access management, and implement appropriate controls to mitigate those risks.

# Trust frameworks

Trust frameworks in information security are sets of standards, policies, and procedures that establish trust between parties in a digital transaction. These frameworks are designed to provide assurance that transactions are secure, reliable, and private.

## **Traditional Identity Exchange Approach:**

The traditional identity exchange approach is a trust framework that involves the exchange of identity information between parties in a transaction.

In this approach, a trusted third party acts as an intermediary to verify and authenticate the identities of the parties involved in the transaction.

This approach is often used in financial transactions, where parties need to be sure of each other's identities before exchanging funds.

## **Open Identity Trust Framework:**

The open identity trust framework is a trust framework that is based on open standards and protocols, such as OpenID and OAuth.

In this approach, users maintain control over their own identities and can choose to share their identity information with other parties as needed.

The open identity trust framework is often used in online services and applications, such as social media platforms, where users need to authenticate their identities to access services and share information with other users.